Computers & Security 171 (2026) 105088

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

An effective approach to typosquatting detection ☆

Francesco Blefari [a], Angelo Furfaro [a,*], Giovambattista Ianni [b], Francesco Aurelio Pironti [a]

[a] DIMES - University of Calabria, P. Bucci 41C, Rende, I-87036, CS, Italy
[b] DeMaCS - University of Calabria, P. Bucci 30B, Rende, I-87036, CS, Italy

ARTICLE INFO

Dataset link: https://github.com/cybers3clab/typosquatting_dataset

Keywords:
TypoSquatting
URL hijacking
Privacy
Phishing

ABSTRACT

Typosquatting is a deceptive practice that involves exploiting typographical errors made by users when manually entering URLs. Although well-known and widely exploited, effective user-side mitigation tools for typosquatting remain scarce. In this work we propose an effective, user-side, typosquatting detection scheme called TypoDS, which combines and improves state-of-the-art typosquatting detection techniques and which is made freely available as a dedicated extension, named TypoAlert, for the popular web browser Chrome. TypoAlert is designed to automatically detect in real-time when a user visits a typosquatted domain and alert the user accordingly. The proposed approach relies on extracting informative features from URLs. These features are converted into quantitative scores, which are then combined by means of a threshold-based scoring function, whose threshold is experimentally determined, to classify a URL as legitimate or typosquatted. The proposed scheme has been validated on a dataset with a substantial number of labeled domains, and its performance has been compared against machine-learning-based classification algorithms. The results show that a TypoDS implementation is able to reach an accuracy of 98.41%.

1. Introduction

Among digital threats, cybersquatting has emerged as one of the most insidious forms of illegal exploitation. This illicit practice involves individuals or entities acquiring the exact domain names that correspond to established trademarks or well-known personalities. In response, many brands acted in the past to address this issue by acquiring or reclaiming their respective domains, thereby mitigating the prevalence of cybersquatting. However, the threat has evolved into a more sophisticated tactic known as *typosquatting*. Typosquatting exploits typographical errors made by users when entering URLs into their browsers. Attackers, known as *typosquatters*, register domains that feature minor spelling variations of legitimate websites, capitalizing on the inevitable user typing mistakes, and on the high likelihood that users overlook subtle differences in URLs within emails or other communications. This attack method is particularly effective when targeting high-traffic domains, as even a small percentage of user errors can redirect a significant amount of traffic to *typosquatted* sites.

Typosquatted sites, i.e. websites whose domain names closely resemble legitimate ones, often host a variety of content designed to generate revenue through advertising. These sites frequently include malicious elements or redirect users to harmful websites. Typosquatted domains are also commonly used to conduct attack campaigns, such as phishing, which aim to steal sensitive information from unsuspecting users. In a phishing campaign, malformed domain names, presented in malicious e-mails or in other communication channels, often go unnoticed by end users.

Several studies indicate that between 10% and 20% of manually entered URLs contain errors (Boutin, 2005). As a result, the average user who mistypes the URL of a popular site faces a 1 in 14 chance of landing on a typosquatted domain Chen et al. (2009). The consequences of typosquatting are substantial: companies suffer from traffic and revenue losses, while users face an ongoing risk of falling victim to online scams.

Despite extensive research in this field over the years (Spaulding et al., 2016) and the development of several prototypical anti-typosquatting tools, practical, real-time solutions to assist users are uncommon. The contributions of this work are the following:

1. We show how to combine and improve multiple anti-typosquatting methods into an integrated framework, called TypoDS;
2. We introduce a set of both numeric and boolean indexes that are used as features for typosquatted domain classification. These indices are combined through a purpose-built mathematical function, whose output is compared against a predefined threshold

☆ Preliminary results concerning this work have been published in Blefari et al. (2024).

* Corresponding author.
E-mail address: angelo.furfaro@unical.it (A. Furfaro).
URL: https://angelo.furfaro.dimes.unical.it (A. Furfaro).

https://doi.org/10.1016/j.cose.2026.105088
Received 27 February 2026; Received in revised form 24 July 2026; Accepted 28 July 2026
Available online 1 August 2026
0167-4048/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

to determine whether a domain is typosquatted. Remarkably, since our method is not based on machine learning (ML), it does not require model training or periodic retraining to cope with concept drift, thereby reducing the maintenance effort of TypoDS to a minimum.

3. We implement our technique-coalescing method in an extension for the Chrome browser, called TypoAlert; besides including our integrated method to detect typosquatted sites, TypoAlert properly manages web site visits and prompts user intervention only whenever necessary, following the known principle of *least user surprise* (Raymond, 2008);
4. We produce an appropriate domain classification dataset of 10000 entries that can be used as a ground truth and as basis for other research works;
5. We assess the quality of our ensemble techniques on the above dataset; our techniques were compared with state-of-art machine learning classification algorithms, and with the lookalike feature of the Google Chrome web browser;we demonstrate that our approach achieves comparable or superior levels of effectiveness.

The rest of this paper is structured as follows: in Section 2 we present previous studies on typosquatting and illustrate the necessary background information. In Section 3 we present our typosquatting detection methodology. Section 4 provides a comprehensive evaluation: it starts with an analysis of current datasets and their shortcomings, followed by a description of our proposed dataset and the related experimental results.

The browser extension TypoAlert is presented in Section 5. Eventually, we draw some conclusions and outline future research directions in Section 6.

2. Background and related work

The general term cybersquatting refers to an as old as the World Wide Web itself. Cybersquatting occurs when some entity (both an individual or a corporation) registers a domain name that is spelled the same as a pre-existing trademark. Money is demanded to the trademark owner in order to release the domain name (Mercer, 2000). Cybersquatting evolved to cyber-typosquatting (or typosquatting, for brevity) when malicious entities started to register domain names slightly different from legitimate names, with the intent of diverting traffic or performing other illegal operations. Nowadays, the term refers not just to domain name typosquatting but also to package typosquatting (Taylor et al., 2020; Neupane et al., 2023) and to other forms of typosquatting over other string entities, like mobile app names, social media names, etc. The phenomenon of typosquatting has been the subject of study and research for at least a quarter of a century: it can be traced back at least to the origin of the term *typosquatter* around 1998. Typosquatting remains a widespread and persistent practice, primarily due to the lack of effective solutions to prevent it (Chen et al., 2009). We will particularly focus on domain name typosquatting, also called, with a slightly different semantic nuance, *URL typosquatting*. Existing research on the topic can be loosely categorized in: *(i)* general analyses and organizational proposals against the domain typosquatting phenomenon; *(ii)* company-centric anti-typosquatting proposals, oriented towards the perspective of companies concerned with typosquatting attacks against their websites; and, *(iii)* user-centric anti-typosquatting research, i.e. tools and methods conceived to reduce risk of users stumbling upon typosquatted sites while browsing the web.

2.1. General studies

One of the earliest large-scale studies conducted on this phenomenon was published by Edelman (2003). The study identified over 8800 registered domains that had subtle typographic variations

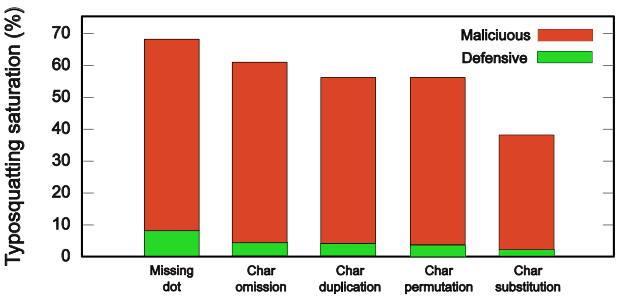


Fig. 1. Saturation of typosquatted domains divided by generation models (Agten et al., 2015).

compared to popular domain names. More than 90% of these domains redirected to sexually explicit content. Additionally, these domains were intentionally designed to make it hard to shut down the offending content. Initially, it was believed that the length of the domain name was a crucial attribute to consider, asserting that over 10% of all possible registered typo domains have less than 10 characters, and thus, shorter domain names were more susceptible to typosquatting (Banerjee et al., 2011). However, this hypothesis was later refuted (Moore and Edelman, 2010), indicating that domains with longer names share a similar probability of being subject to typosquatting. Indeed, many of the statements made in older studies about typosquatting have been subsequently revised or reformulated by more recent research. This underscores the complexity and variability of the typosquatting phenomenon in the contemporary digital landscape. Similarly, the popularity of domain names was originally seen as a factor closely related to typosquatting, suggesting that a decrease in the popularity of a domain implied a lower likelihood of typosquatting on it (Banerjee et al., 2011). This assumption has also been revisited, with a subsequent study (Szurdi et al., 2014) identifying a shift in typosquatters' behavior: around 95% of typosquatted domains now targets less popular domains. These conceptual shifts can be partially attributed to the early stage of the phenomenon, where attention was primarily focused on the most popular domains, often characterized by short names. Indeed, there has been a widening of focus towards less known domains with more extensive names due to the saturation of typo-domains over time.

Most research into the phenomenon has historically followed a two-phase methodology: *(i)* identification of a set of victim domain names, and *(ii)* generation of a list of potential typosquatted variants of these domain names. Various models for generating typosquatted domains have been proposed, necessitating a comprehensive understanding of their underlying logic. Given a domain such as `www.example.com`, five primary models have been identified (Wang et al., 2006): *Missing-dot* typos, *Character-omission* typos, *Character-permutation* typos, *Character-substitution* typos, and *Character-duplication* typos. To assess the awareness of these diverse models among typosquatters and legitimate domain owners, a subsequent study (Agten et al., 2015) scrutinized registered domains generated according to each of the five described models, evaluating their saturation. This research has been instrumental in delineating the landscape of domain occupation within the realm of typosquatting. Moreover, it has provided valuable insights into the level of awareness regarding various typo domain generation models among distinct online entities.

Fig. 1 shows the *typosquatting saturation* of each generation model, i.e. the percentage of active domains among all typosquatting domains generated according to a given model starting from the list of authoritative domains employed in Agten et al. (2015). It can be seen that the character substitution model exhibits the least popularity, whereas the missing-dot model stands out as the most prevalent: approximately 70% of typosquatted domains generated using the missing-dot model are registered. Note that this percentage includes defensive registrations,

i.e. the preventive acquisition of typosquatted domain names by owners of legitimate websites. Both malicious and defensive registrations mirror the saturation trends. This implies that both attackers and defenders share a similar perception regarding the typosquatted domains deemed worthy of registration.

Existing models were extended by introducing additional approaches in Banerjee et al. (2008). These include:

- *1-mod-inplace*: This method involves systematically replacing, one at a time, all characters in the domain name with every possible letter of the alphabet;
- *1-mod-deflate*: This approach entails removing, one at a time, all characters from the domain name;
- *1-mod-inflate*: This method involves adding a character to the domain name, systematically considering all possible characters.

All these generation models are fundamentally rooted in the concept of edit distance or Levenshtein distance (Levenshtein, 1965; Damerau, 1964). This metric quantifies the similarity between two strings by determining the minimum number of operations (insertions, deletions, or substitutions) needed to transform one string into another. Over time, it has become a crucial parameter for evaluating the similarity of typo domains to the original domain. However, when considering the character permutation generation model, the more appropriate reference is the Damerau–Levenshtein distance (Navarro, 2001). This distance metric differs from the classic Levenshtein distance by incorporating the operation of transposition between characters, in addition to insertion, deletion, and substitution operations. Research indicates that 99% of typosquatted sites exhibit a Damerau–Levenshtein distance of one from their target domains (Spaulding et al., 2016).

The prevalence of typosquatting exhibits a correlation with the popularity of the corresponding Top Level Domain (TLD). Notably, the `.com` TLD, representing a substantial portion of registered domains, demonstrates a heightened susceptibility to typosquatting as observed in Spaulding et al. (2016). Additionally, instances of typosquatting extend beyond `.com`, with users inadvertently accessing www.whitehouse.com instead of the official White House website (www.whitehouse.gov) (Agten et al., 2015). Country-code domains, such as `.it` and `.en`, are particularly vulnerable to typosquatting. This can be attributed in part to the absence of a Uniform Domain-Name Dispute-Resolution Policy (UDRP) for these TLDs, rendering cross-TLD domain name similarities attractive to typosquatters due to reduced legal ramifications.

Beyond the structural and regulatory factors that influence typosquatting, user behavior also plays a crucial role in identifying such domains. A key behavioral indicator is bounce rate, which measures the proportion of visitors who leave a site shortly after arriving. Khan's study (Khan et al., 2015) delves into the temporal aspects of user behavior, revealing that individuals landing on typosquatted domains typically recognize the error and redirect to the authentic domain within approximately 33 s. This temporal element is critical in distinguishing genuine typosquatting instances from innocent similarities, preventing genuine misinterpretations such as mistaking `nhl.com` for `nfl.com`. Using this type of feature for detecting typosquatting however requires to continuously maintain some form of machine learning model, and does not fit appropriately if a typosquatted domain has to be detected in real-time.

2.2. Company and user-centric anti-typosquatting tools

Several tools have been developed in the past, with the aim to mitigate the impact of typosquatting both on companies protecting their web resources, and on the users' web-surfing experience.

The pioneer Strider Typo-Patrol tool (Wang et al., 2006) was meant for discovering large typosquatting campaigns. It employed a multi-faceted approach, incorporating: *(i)* a Typo-Neighborhood Generator to produce sets of URLs with potential typos, *(ii)* a Typo-Neighborhood Scanner to actively analyze domains and record information such as third-party URLs and page content, and *(iii)* a Domain-Parking Analyzer for in-depth analysis of typosquatted domains. The same work proposed Strider URL Tracer, an application devised to allow website owners to monitor typosquatted domains targeting their sites.

A comprehensive and relatively recent analysis of typosquatting domain registrations within the `.com` TLD can be found in Szurdi et al. (2014). The analysis was conducted using the Yet Another Typosquatting Tool (YATT). The adopted methodology included the usage of passive data sources (e.g., `.com` zone files, domain blacklists), candidate typo generation, active analysis through Whois, DNS, and web crawling, and domain categorization based on Damerau–Levenshtein distance and registration information.

In the realm of user-centric approaches, we mention the now defunct iTrustPage (Ronda et al., 2008) extension for Firefox, which provided automated identification of legitimate web pages through an Interactive Page Validation process. The approach utilized user input and external sources such as search engine results, including whitelists and local caches. The Anti Typosquatting Tool (ATST) (Chen et al., 2009) was a browser extension featuring *(i)* a User Customized Local Repository for monitoring popular domains, *(ii)* an Edit-distance Computation Module employing the Damerau–Levenshtein distance for typosquatted domain checks, and *(iii)* a User Customized Local Repository Update Module for dynamic updates based on user interactions.

The Stop URL Typo-squatting (SUT) approach was proposed in Banerjee et al. (2011). SUT addresses the broader issue of detecting what the authors define *phony* websites, whose domain name is not necessarily typosquatted. This solution adopted a two-level structure integrating autonomous modules for network-level criteria (SUT-net) and site popularity assessment (SUT-pop). SUT-net assessed URL features such as length, redirects, Top-Level Domain, page size, pop-ups, and keywords, while SUT-pop leveraged Google search results to evaluate domain legitimacy.

It is also worth mentioning TypoWriter (Ahmad et al., 2019), which anticipates likely domain variations using Recurrent Neural Networks trained on DNS logs.

Other approaches include the possibility of alerting the user with blatantly different visual displays, in case a counterfeited site is accessed, using so called dynamic skins (Dhamija and Tygar, 2005).

Unfortunately, at the time of writing, all of the above mentioned tools are no longer available. In Table 1 a synthesis of the discussed above works is provided.

2.3. Edit distances

An edit distance is a metric that quantifies the dissimilarity between two strings. Given two strings, s_1 and s_2, we can transform s_1 into s_2 through a sequence of four possible operations:

- **Insertion:** Adding a single character at any position within the string. For instance, transforming s_1 = "`car`" into s_2 = "`cart`" requires the insertion of the character '`t`' at the end of the string.
- **Deletion:** The inverse operation of insertion, in which a single character is removed from the string. For example, transforming s_1 = "`color`" into s_2 = "`colo`" requires the deletion of the character '`r`'.
- **Substitution:** Replacing a character at a specific index with a different character. In some models, this is mathematically treated as a deletion followed by an insertion at the same position. For example, transforming s_1 = "kitten" to s_2 = "`sitten`" involves substituting the initial '`k`' with an '`s`'.
- **Transposition:** Swapping the positions of two adjacent characters. While excluded from the classic Levenshtein distance, it is a core operation in the Damerau–Levenshtein distance, specifically introduced to reflect common human typographical errors. For instance, transforming s_1 = "`teh`" into s_2 = "`the`" requires a single transposition of '`e`' and '`h`'.

Table 1
Summary of general studies on domain typosquatting.

Study	Research focus	Main findings
Edelman (2003)	Large-scale measurement of domain typosquatting	Identified more than 8800 typo-related registered domains. Over 90% redirected users to sexually explicit content. Highlighted difficulties in identifying responsible parties and removing abusive content.
Banerjee et al. (2011)	Impact of domain characteristics on typosquatting	Investigated the relationship between domain length and typosquatting exposure. Reported that shorter domains were more frequently targeted.
Moore and Edelman (2010)	Reassessment of domain characteristics	Showed that longer domains can exhibit comparable typosquatting rates, challenging the assumption that domain length is the main predictor of risk.
Szurdi et al. (2014)	Effect of domain popularity	Demonstrated that typosquatting also affects less-popular domains, showing that measurements limited to highly ranked domains underestimate the phenomenon.
Wang et al. (2006)	Typo-generation models	Defined five common typo-generation models: missing-dot, character-omission, character-permutation, character-substitution, and character-duplication.
Agten et al. (2015)	Saturation of typo-generation models	Measured the fraction of generated typo variants that were registered. Missing-dot variants showed the highest saturation (about 70%), while character-substitution showed the lowest. Saturation includes both malicious and defensive registrations.
Banerjee et al. (2008)	Extension of typo-generation strategies	Introduced additional models based on one-edit operations: character replacement (1-mod-inplace), deletion (1-mod-deflate), and insertion (1-mod-inflate).
Levenshtein (1965), Damerau (1964)	String similarity metrics	Established edit distance as a fundamental metric for measuring similarity between legitimate domains and typo variants. Damerau–Levenshtein distance additionally captures character transpositions.
Spaulding et al. (2016)	Distance-based analysis and TLD impact	Found that approximately 99% of typosquatted domains have a Damerau–Levenshtein distance of one from the target domain. Also highlighted the higher exposure of popular TLDs such as .com.
Khan et al. (2015)	User behavior analysis	Showed that users reaching typosquatted domains typically identify the mistake and return to the legitimate website within approximately 33 s. Bounce-rate features can help detection but require continuous model maintenance.

Table 2
Examples of edit operations applied to domain names.

Operation	Legitimate domain	Modified domain	Edit
Insertion	`google.com`	`gooogle.com`	Insert `o`
Deletion	`amazon.com`	`amzon.com`	Delete `a`
Substitution	`paypal.com`	`paypa1.com`	Replace `l` with `1`
Transposition	`facebook.com`	`faceboko.com`	Swap adjacent characters

By considering different combinations of edit operations, several distinct string distance metrics can be defined. The Hamming distance (Hamming, 1950) accounts exclusively for substitutions and therefore requires strings of equal length. The Jaro distance (Jaro, 1989) is designed for record linkage tasks and primarily captures character transpositions, while also tolerating limited mismatches. The Longest Common Subsequence (LCS) metric (Wagner and Fischer, 1974) is based on insertions and deletions, measuring similarity through the length of the longest shared subsequence.

The Damerau (1964) edit distance extends the basic edit model by including insertions, deletions, and substitutions. The most comprehensive formulation is the Damerau–Levenshtein (DL) distance (Damerau, 1964), which incorporates all four fundamental operations: insertions, deletions, substitutions, and adjacent transpositions.

The *DL* distance was selected over simpler edit distances because it models human typing errors more realistically, specifically by accounting for transpositions. Cited literature indicates that a distance of $DL = 1$ covers the vast majority of typosquatted domains. Other metrics, such as the *Kullback–Leibler divergence*, were excluded as they are designed for comparing probability distributions and do not yield meaningful results in the context of string-based domain name comparison. In Table 2 examples are present for each operations.

2.4. Recent work

Beyond traditional edit-distance-based domain generation, typosquatting detection has increasingly leveraged machine-learning and artificial-intelligence techniques. TypoSwype (Joon Sern and Gui Peng David, 2021) represents domain names as keyboard-layout-aware images and uses convolutional neural networks to learn a similarity representation that accounts for the spatial proximity of keys. This approach improves over purely string-based comparison, but it requires model training and a representative dataset (Joon Sern and Gui Peng David, 2021).

Language-model-based techniques have also been proposed for the broader problem of generated squatting domains. For instance, PhishReplicant (Koide et al., 2023) uses linguistic similarity between domain names to identify generated squatting domains. While this direction is promising for detecting more complex and previously unseen patterns, it relies on learned models and targets a broader class of domain abuse than typo domains alone (Koide et al., 2023).

More recently, LLM-based methods have been investigated for typosquatting detection. Welch (2025) fine-tuned language models on character-level transformations and pattern-based heuristics, reporting high classification accuracy. However, the computational cost,

Table 3
Comparison of representative recent approaches for typosquatting and related domain-squatting detection. Reported results are not directly comparable because the studies use different datasets, threat models, and evaluation protocols.

Work	Year	Scope	Detection method	Reported outcome	Main limitations
TypoSwype (Joon Sern and Gui Peng David, 2021)	2022	Domain typosquatting	Keyboard-layout-aware image representation and CNN-based similarity learning	Improves typo-squatting detection over Damerau–Levenshtein-based comparison while preserving target-domain classification accuracy	Requires model training, suitable training data, and computational resources for inference
PhishReplicant (Koide et al., 2023)	2023	Generated squatting domains	Language-model-based analysis of linguistic similarity between domain names	Detects generated squatting domains that may not be captured by manually specified typo-generation rules	Addresses a broader squatting problem; learned-model behavior depends on training data and may be less transparent
Welch (2025) (preprint)	2025	Domain typosquatting	Fine-tuned LLM using character-level transformations and pattern-based heuristics	Reports up to 98% accuracy for the best evaluated model configuration	Requires fine-tuning, model deployment resources, and periodic maintenance to address evolving attack patterns
TypoDS	2026	Domain typosquatting	Interpretable feature-based scoring combining domain reputation, defensive redirects, parking signals, search-engine evidence, and DL-based candidate targets	98.41% accuracy on the proposed balanced dataset; evaluated against ML baselines and Chrome's lookalike mechanism	Depends on the availability and stability of external online sources; currently focuses on domains close to candidate targets

model maintenance, and reproducibility requirements of LLM-based approaches may limit their adoption in lightweight real-time browser-side tools (Welch, 2025).

Table 3 summarizes representative recent approaches and positions TypoDS with respect to their detection paradigm, reported outcome, and practical limitations.

The comparison highlights the design rationale of TypoDS. Unlike learning-based approaches, TypoDS does not require a training phase or periodic retraining to operate. Instead, it combines interpretable and lightweight signals that can be obtained at navigation time, including blacklist and whitelist membership, defensive redirects, parking-page evidence, Damerau–Levenshtein-based candidate targets, and search-engine signals. This design makes TypoDS particularly suitable for real-time browser-side deployment, while retaining a feature-based decision process that can be inspected and updated without retraining a predictive model.

Algorithm 1 Prefiltering Algorithm

Input: dn, the domain name to analyze
Output: $(blacklisted, whitelisted, green_typo)$

$(blacklisted, whitelisted, green_typo) \leftarrow (0, 0, 0)$
$vd \leftarrow follow_forward(dn)$
if $dn \in BL$ or $vd \in BL$ **then**
  $blacklisted \leftarrow 1$
else if $n \in WL$ **then**
  $whitelisted \leftarrow 1$
else if $vd \in WL$ **then**
  $green_typo \leftarrow 1$
end if
return (*blacklisted*, *whitelisted*, *green_typo*)

3. Detection scheme

The proposed detection scheme, called TypoDS, combines several anti-typosquatting techniques and can be implemented as a plugin for real-time monitoring and detection of typosquatted web domains. In the following, let us assume we are given a domain name (*dn*), a possible typosquatting attempt. We determine an *alert value* (*av*) for *dn*.

av is built on the basis of dynamic knowledge about *dn*, e.g. reputation on web search engines, and static knowledge, e.g. a whitelist of *safe* domains and a blacklist of *malicious* domains. The typosquat detection process takes place in three main steps: (a) *prefiltering*; (b) *feature extraction*; (c) *classification*.

The first step performs a skimming pass, by checking whether the considered domain is present in some particular domain list. In the second step, we extract informative features about *dn*. The last phase computes *av* based on the values of informative features. Fig. 2 presents a schematic overview of these steps, including the indicators and features computed by the algorithms. The three steps are described in detail below.

3.1. Prefiltering

The prefiltering step is summarized in Algorithm 1. First, basic checks are performed in order to classify some of the more simple cases by computing three basic binary features, namely *blacklisted*, *whitelisted* and *green_typo*. We assume this check has priority over more in-depth analysis, i.e., if one of the basic features is set to 1, the subsequent feature extraction step is skipped. The first two features, *blacklisted* and *whitelisted*, are computed by checking if *dn* belongs or not to a whitelist (*WL*) of benign websites or to a blacklist (*BL*) of dangerous websites. *BL* is populated using BlackBook (Stampar, 2025), a continuously updated historical blacklist that contains a wide range of domains known for their malicious nature. This blacklist is kept up-to-date by means of an automated monitoring process based on a set of reliable sources such as WHOIS data, HTTP analysis, and so on. In particular, Blackbook includes new entries reported on public lists of harmful URLs, such as CyberCrime info center (Interisle Consulting Group, 2025), URLhaus (abuse.ch, 2025), ScumBots (signalblur, 2025), and ViriBack (ViriBack, 2025). Although the entries contained in the black list are more generally attributed to dangerous websites, a number of typosquatted domains is present. A match of an analyzed domain with one contained in *BL* classifies the domain as malicious, either because it is a known typo domain or because it is dangerous. Accordingly, if $dn \in BL$ then the *blacklisted* feature is set to 1 and dn will get the maximum alert value (see below).

WL is constructed by joining a Top Domain Repository (*TDR*) together with a user-built custom whitelist (User Domain Repository, *UDR*), i.e., $WL = TDR \cup UDR$. TDR includes domains that can be considered reliable and authentic. TDR is dynamically built by coalescing the list of the 1000 most visited sites globally and the lists of the 1000 most visited sites for each country (DataForSEO, 2025), for

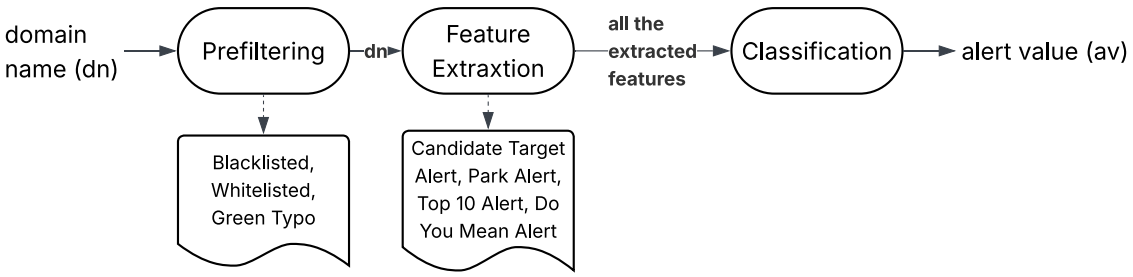


Fig. 2. TypoDS diagram. Starting from a domain name, each phase is executed once to compute the features used for classification. If the pre-filtering phase successfully identifies the domain as blacklisted or whitelisted, the feature extraction phase can be skipped.

a total of around 32000 distinct domain names[1]. The primary rationale behind systematically including these top-visited sites is to shield users from accidental typing errors, a vulnerability frequently exploited by typosquatting attacks. Malicious actors routinely register domains that are typographically similar to high-traffic websites to intercept navigational mistakes. By explicitly whitelisting the canonical domains, any slight deviation caused by a typo will fail to match the *TDR*, thereby preventing the connection to the imposter site. While a high traffic ranking does not intrinsically guarantee absolute security against all web-based threats (such as compromised legitimate servers), it serves as a robust heuristic for domain authenticity. It ensures that the domain is the legitimate, intended destination, effectively protecting the user from deceptive impersonation.

As elements of WL are assumed to be safe domains, for a domain name $n \in WL$ *whitelisted* is set to 1 and dn gets the minimum alert value. *TDR* cannot be modified by the user, which can instead customize the complementary *UDR* whitelist, which is initially empty, where she/he can add those domains she/he considers to be safe.

The *green_typo* feature takes into account the presence of defensive registrations. Legitimate website owners are indeed used to register potential typo domain names. Typically, when a typosquatted domain is of the defensive type, a redirect is placed, pointing to the legitimate domain at hand. For example, at the time of writing, `faceboook.com` redirects to `facebook.com`. We call *green typos* those domain names which were *defensively* registered to prevent malicious exploits of typos.

This feature is dynamically computed by visiting *dn* and following possible redirects by means of the function $follow_forward$. Let $vd = follow_forward(dn)$ be the domain name eventually reached from dn after following a potential chain of redirects. The membership of vd in either *WL* or *BL* is then checked and used for evaluating whether the URL corresponding to dn points to the "right" website even when starting from a "wrong" domain name. The function $follow_{forward}$ takes a domain name dn as input and performs an HTTP request to the corresponding URL. If the server responds with an HTTP 302 (Found) redirect, the function extracts the value of the Location header and repeats the request for the redirected URL. This process continues until a response other than HTTP 302 is received or a predefined maximum number of redirects is reached to prevent infinite redirect loops. The domain name of the final destination is returned as vd.

In summary, the prefiltering step considers the input domain dn and searches it within *BL* first. If $dn \in BL$ or $vd \in BL$ the *blacklisted* feature is set to 1. In the case $dn \in WL$ the domain is assumed to be safe and the feature *whitelisted* is set to 1. If $dn \notin WL$ but $vd \in WL$ the visited domain is also deemed as safe and then the $green_typo$ feature is set to 1.

[1] The *TDR* used in this work dates back to January 16, 2024.

3.2. Feature extraction

This step is performed only if the previous step does not set any feature to 1, and extracts four additional informative features from the domain name dn. In this phase we rely on information dynamically obtained from the internet by several means. The additional features are the following:

- Parking Alert (*PARKA*);
- Candidate Target Alert (*CTA*);
- Top 10 Alert ($T10A$);
- Did You Mean Alert (*DYMA*).

Parking alert

It can commonly happen that *dn* is a typosquatted registration and hosts a *parked* website. A parked domain is a domain name that is registered but not actively being used for a website or specific online service. Instead, a parked website typically displays a placeholder page, advertisements, or a message indicating that the domain is reserved for future use. *PARKA* is a binary feature that is based on a multi-language analysis. One looks in the web page of the reached domain vd for the occurrence of key-phrases usually present in parked web pages or simply checks if the reached domain vd is a known hosting resource for parked domains. According to the analysis of key-phrases, the *PARKA* indicator is set to 1 if the domain dn is evaluated as a parking domain, otherwise it is set to 0. In order to implement *PARKA*, sophisticated techniques like, e.g., small neural networks analyzing the screenshot of the landed domain, are possible.

However, we found that a much simpler heuristic provides satisfactory performance for the purposes of this work. Specifically, we compare the textual content of the landing page against a manually curated, multilingual dataset of phrases commonly found on parked domains (e.g., "This domain can be yours"). In addition, we verify whether the final visited domain belongs to a manually compiled list of well-known domain parking providers (e.g., `godaddy.com`). If either of these conditions is satisfied, the *PARKA* feature is set to 1; otherwise, it is set to 0. While this heuristic is computationally inexpensive and proved sufficiently effective in our experiments, it is not without limitations. In particular, it may produce false negatives when parked domains use uncommon or highly customized landing pages, display little or no textual content, rely on client-side rendering that is not captured during analysis, or are hosted by parking providers that are absent from our curated list. This approach may also fail to identify parked domains whose landing pages consist mainly of an image displaying a commercial offer to purchase the domain, as no machine-readable text is available for keyword matching. Consequently, although the proposed implementation is adequate for this study, more sophisticated detection techniques could improve robustness and will be investigated in future work.

Algorithm 2 Candidate Target Extraction

Input: dn, the domain name to analyze
Output: CT

$CT \leftarrow \emptyset$
for $w \in WL$ **do**
  if $distance_{\mathrm{DL}}(dn, w) = 1$ **then**
    $CT \leftarrow CT \cup \{w\}$
  end if
end for
$Q \leftarrow \mathrm{search}(dn)$
if $Q.DYM \neq null$ and $distance_{\mathrm{DL}}(dn, Q.DYM) = 1$ **then**
  $CT \leftarrow CT \cup \{Q.DYM\}$
end if
for $t \in Q.T10$ **do**
  if $distance_{\mathrm{DL}}(dn, t) = 1$ **then**
    $CT \leftarrow CT \cup \{t\}$
  end if
end for

Candidate targets

The remaining three features are based on the computation of a set of *candidate targets* (CT) which are relevant for a domain. Given a domain name dn, CT is built by filtering domains from WL and by selecting domains obtained by querying a search engine S with dn as the search keyword. The steps for building CT are detailed in Algorithm 2. In particular, the domains extracted from querying S are:

(*i*) the ranked list of the top 10 domain names (*T10*) resulting by querying S with dn as the search keyword;
(*ii*) the *"Did you mean?"* (*DYM*) result, whenever it is available. The *DYM* domain is the domain name suggested by S as the inferred correct search keyword.

CT is then built by taking all the domain names, having *Damerau–Levenshtein* distance from dn equals to 1, taken from *(i) WL*; *(ii) T10*; and *(iii)* the *DYM* domain, when available.

Accordingly, the *Candidate Target Alert* (*CTA*) is a binary feature which is set to 0 when the CT list is empty, otherwise it is set to 1. This feature tells whether the domain name *dn* is a potential typosquat of at least one domain.

Top 10 and did you mean alerts

T10A and *DYMA* are special indicators derived from the results obtained from the computation of the candidate targets for the domain *dn*. In particular, the *T10* list and the *DYM* domain, if present, are employed.

For each $ct \in CT$, the $T10A_{ct}$ indicator first evaluates the relative ranking of dn and ct in the search engine's list *T10* obtained by searching dn as keyword. Then, the $T10A_{ct}$ score is formed by comparing the contribution given by the position of dn and ct within *T10*:

$$T10A_{ct} = 0.1 \cdot (\mathrm{rank}(dn) - \mathrm{rank}(ct))$$

where $rank(n) \in \{0, \ldots, 10\}$ is defined as:

$$rank(n) = \begin{cases} pos(n) & \text{if } n \in T10 \\ 10 & \text{otherwise} \end{cases}$$

and $pos(n)$ is the position where n appears in *T10*. Specifically, $T10A_{ct}$ is proportional to the difference between the position of dn and that of ct, if they both appear in *T10*.

If ct occurs in $T10$ but *dn* does not, $T10A_{ct}$ scores a positive value (up to 1.0 if ct is the first element of $T10$) suggesting that *dn* is a probable typosquat of *ct*.

The *DYMA* indicator leverages the concept of domain popularity based on search engine suggestions. It uses suggestions coming from the search engine S about a possible typing error in *dn*. For each $ct \in CT$, the value $DYMA_{ct}$ is defined as:

$$DYMA_{ct} = \begin{cases} 1 & \text{if } DYM = ct \\ 0 & \text{otherwise} \end{cases}$$

i.e. it is set to 1 if S suggests ct when searching for *dn*.

The values of $T10A$ and *DYMA* for *dn* are then respectively computed as:

$$T10A = T10A_{ct^*}, \quad DYMA = DYMA_{ct^*}$$

where:

$$ct^* = \arg\max_{ct \in CT} \{T10A_{ct} + DYMA_{ct}\}$$

i.e. $ct^* \in CT$ is the domain name for which the sum $T10A_{ct}+DYMA_{ct}$ is highest. The choice of using the max value of this sum is meant to minimize the number of false negatives: $T10A$ and *DYMA* will be 0 only if there is no candidate domain at all which can be possibly a victim of typosquatting because of dn. We set $T10A$ and *DYMA* to 0 whenever CT is empty.

Algorithm 3 summarizes the features extraction step.

Algorithm 3 Features Extraction

Input: dn, the domain name to analyze
Output: $(blacklisted, whitelisted, green_typo, CTA, PARKA, T10A, DYMA)$

if $blacklisted \neq 0$ or $whitelisted \neq 0$ or $green_typo \neq 0$ **then**
  return $(blacklisted, whitelisted, green_typo, 0, 0, 0, 0)$
end if
$CT \leftarrow extractCT(dn)$
$PARKA \leftarrow evaluate_{\mathrm{PARKA}}(vd, keyphrases)$
if $CT \neq \emptyset$ **then**
  $CTA \leftarrow 1$
else
  $CTA \leftarrow 0$
end if
$max, T10A, DYMA \leftarrow -1$
for ct **in** CT **do**
  $T10A_{ct} \leftarrow evaluate_{T10}(dn, ct, P)$
  $DYMA_{ct} \leftarrow evaluate_{DYM}(ct, P)$
  if $max < T10A_{ct} + DYMA_{ct}$ **then**
    $T10A \leftarrow T10A_{ct}$
    $DYMA \leftarrow DYMA_{ct}$
    $max \leftarrow T10A_{ct} + DYMA_{ct}$
  end if
end for
return $(0, 0, 0, CTA, PARKA, T10A, DYMA)$

3.3. Classification

Once computed, all of the above features can be fed to a feature-based classification algorithm in order to establish whether *dn* is a *Typo* or a *NotTypo*. There are several well-known machine learning classification algorithms that can be effectively exploited for this goal. Here, we propose an empirical classification algorithm which is not based on machine learning techniques and does not require any preliminary learning step.

The classification algorithm we propose is based on a function that, given a domain name *dn* and the corresponding values for the above identified features, computes an *alert value*, $av \in [0, 7]$, and compares it with a threshold value th^*. If av is greater than th^*, *dn* is classified as *Typo* and as *NotTypo* otherwise. The actual value of th^* we used in the evaluation campaign is 2.1. As discussed in Section 4, this threshold was selected to optimally balance recall and precision: it sits just above

$$av = \begin{cases} 0 & \text{if } whitelisted + green_typo > 0 \\ 7 & \text{if } blacklisted = 1 \\ 2 + PARKA + CTA + \max_{ct \in CT} \left\{ (T10A_{ct} + DYMA_{ct}) \right\} & \text{otherwise.} \end{cases}$$

Box I.

Table 4
Excerpt from the typosquatting data feed dataset of grouped WHOIS records for co-registered domains.

Group	Member	Domain	Registrant Org.	Country	Registrar	Created (UTC)
1	1	tiny.delivery	Private by Design, LLC	United States	Porkbun LLC	2020-08-23 07:43:48
1	2	tiny.observer	Private by Design, LLC	United States	Porkbun, LLC	2020-08-23 07:43:50
1	3	tiny.plumbing	Private by Design, LLC	United States	Porkbun LLC	2020-08-23 10:22:43
2	1	lykes.agency	Domains By Proxy, LLC	United States	GoDaddy.com, LLC	2020-08-23 15:16:02
2	2	lykes.social	Domains By Proxy, LLC	United States	GoDaddy.com, LLC	2020-08-23 15:16:00
2	3	lykes.media	Domains By Proxy, LLC	United States	GoDaddy.com, LLC	2020-08-23 15:15:59

the minimum alert value attainable when $CT \neq \emptyset$, cleanly separating domains with no suspicious signals from all others.

The value of *av* is computed according to the following formula in Box I.

If either the *whitelisted* or the *green_typo* feature is set to 1 for *dn*, *av* takes its minimum value, i.e. 0, suggesting that *dn* is not a typosquat. Similarly, if *blacklisted* is 1, *av* gets its maximum value, i.e. 7. In all other cases, *av* ranges between 1 and 6.

4. Evaluation

To ensure the robustness of our detection framework, we first conducted an extensive evaluation of existing datasets, including the *WhoisXMLAPI* (WhoisXMLAPI, 2025) and Anvilogic's *T5 Typosquat Training Dataset* (Anvilogic, 2025) collections. Our analysis revealed structural and qualitative limitations that necessitate a specialized dataset tailored for active, real-time phishing detection. For these reasons we had to build an up-to-date dataset tailored to our evaluation purposes.

4.1. WhoisXMLAPI dataset

The dataset obtained by the *Typosquatting Data Feed* service of *WhoisXMLAPI* is organized into groups of *co-registered* domains that share similar WHOIS attributes, such as registrant identity, registrar, and registration timestamps. Each row represents a single domain and includes metadata such as registrant name and organization, country and city, registrar, name servers, registration status, and lifecycle dates (creation, update, and expiration). Domains belonging to the same *group_number* are considered related instances (e.g., different TLDs or variants) for the same underlying entity or brand, and the field *total_no_of_grp_members* encodes the size of each group. This structure could enable the analysis of registration patterns across domain portfolios, for example, to detect coordinated domain registrations that may indicate brand protection strategies or, conversely, suspicious bulk registrations. An illustrative extract of the dataset (for groups 1 and 2) is reported in Table 4.

A preliminary evaluation of this dataset revealed significant issues in using it to validate real-time typosquat detection frameworks. Domains belonging to the same group exhibit a certain level of similarity which does not comply with the definition of typosquat considered in this work. The records are generated using an algorithm that samples values from a list of common words to introduce variations into the original TDL. Another issue is related to the *liveness* of the target site, i.e. the actual presence of a web server responding to HTTP(S) requests directed to the corresponding IP address. The fact that the vast majority of referenced domains were unreachable posed a critical barrier. Even when utilizing the most recent snapshots from the provided repositories, no active domains could be resolved, thereby precluding any analysis grounded in live website-derived attributes as in the case of TypoDS.

Furthermore, the dataset presents a notable lack of similar candidates, specifically characterized by a *DL* distance of 1. To facilitate a comparative testing environment, we were forced to artificially relax our classification constraints to allow DL up to 5 and bypass mandatory online availability checks. However, such adjustments introduce a significant bias and deviate from the rigorous operational conditions TypoDS is designed to work with. These limitations are further compounded by a drastic reduction in usable data; after filtering for quality and relevance, the initial pool of 1567 entries yielded fewer than 80 valid domains. The resulting sample size is insufficient to provide the statistical power and/or the qualitative diversity required to draw meaningful conclusions regarding the efficacy of a real-time, feature-rich detection model.

4.2. T5 typosquat training dataset

The *Anvilogic* dataset provides a significantly larger volume of data, structured as a table of triples following the schema: $\langle PotentialTypo, RealWebsite, Label \rangle$. In this format, the binary *Label* indicates whether the *Potential Typo* domain constitutes a typosquatting attempt against the associated *Real Website*. Notably, a single *Potential Typo* may appear in multiple tuples paired with different *Real Websites*. An illustrative extract is provided in Table 5.

Despite its scale, integrating this dataset into our pipeline revealed a fundamental methodological misalignment. The dataset's relational configuration contrasts with our framework's objective of performing autonomous, absolute domain-level classification. To bridge this gap, extensive preprocessing was necessary to resolve label conflicts. Specifically, we defined the set *Typo* of typosquatting domains by including any candidate that appears as the first element in at least one triple where the *Label* is `true`. This approach allowed us to consolidate instances where a single *Potential Typo* was assigned inconsistent labels depending on the specific reference website it was evaluated against.

Furthermore, our preliminary analysis suggests significant concerns regarding the underlying data curation. As illustrated by the distribution of the *DL* distances (see Fig. 3), there is a suspicious correlation between lexical distance and labels, i.e. the majority of triples for which *Label* is `true` exhibits a $DL \leq 6$ between *Potential Typo* and *Real Website*, while nearly all the other cases are characterized by a $DL > 6$. This distribution implies that the dataset may be governed by a simplistic distance threshold rather than the complex lexical or

Table 5
Example of tuples from the Anvilogic's T5 typosquat training.

Potential typo	Real website	Label	Interpretation
imgku-io.xn–9et52u	imhd.io	0 (false)	No typosquatting relation
sciquest.farm	mapquest.com	0 (false)	Unrelated or non-similar domains
gnome-org.eus	gnome.org	1 (true)	Typographical variant of the legitimate domain
199666.xyz	6pm.com	0 (false)	Different structure and branding
acsbapp-com.website	acsbapp.com	1 (true)	Addition of a fake segment ("-com")
sol-casino-club.com	palmplaystore.com	0 (false)	No relevant similarity
bc-sync-com.kn	bc-sync.com	1 (true)	Addition of supplementary TLD
bc-sync-com.kn	shb-sync.com	0 (false)	Different TLD and prefix

Table 6
Detailed evaluation of the T5 typosquat training dataset: Confusion matrix and derived classification metrics.

(a) Confusion Matrix

TN	FP	FN	TP
0 (0.0%)	1 (0.13%)	61 (7.74%)	726 (92.13%)

(b) Classification Metrics

Accuracy	Precision	Recall	F1-Score
0.9213	0.9986	0.9225	0.9590

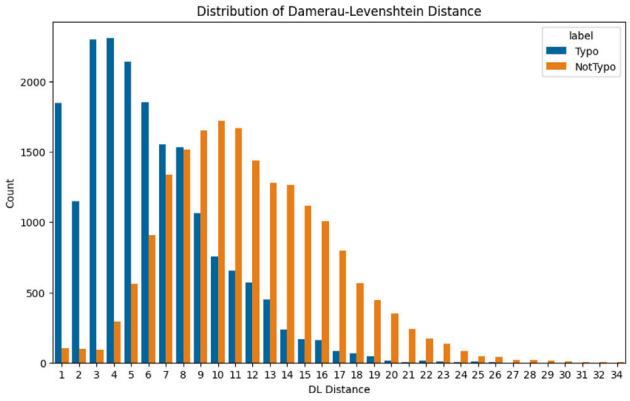


Fig. 3. Distribution of *Typo* and *NotTypo* domains into the T5 Typosquat Training Dataset.

structural properties found in sophisticated phishing campaigns. Consequently, testing against such a biased benchmark may yield overly optimistic performance metrics that do not reflect the nuance of real-world threats.

Moreover, because the vast majority of potential typosquatting domains were inactive at the time of the analysis, the algorithm must disregard domain availability. Consequently, all fields dependent on live data were assigned a null value of 0. Despite these limitations, we conducted a rigorous evaluation by relaxing the active connection constraint.

After the pre-processing phase we obtained a dataset of 1926 entries (1824 *Typo* and 102 *NotTypo*). Using these data, TypoDS achieved an accuracy of 0.9213, a precision of 0.9986 and an F1-score of 0.9590. These results are summarized in Tables 6(a) and 6(b).

4.3. Custom dataset

The discrepancies highlighted above demonstrate that existing datasets are largely composed of historical or dormant data. Since our detection scheme relies on site availability and real-time feature extraction, we developed a custom dataset consisting of active, verified domains. This ensures that the evaluation reflects the actual performance of the tool in a production environment where the "upness" of a site is a primary indicator of its threat level.

In order to rigorously validate the proposed detection scheme and to assess its performance, we needed a dataset of domain names with a known classification (ground truth). At the time of writing, no comprehensive datasets of typosquatted domains with corresponding classifications were available. Therefore, we built from the ground up a dedicated validation dataset, named *TS*. *TS* is a list of records, each represented as a pair (dn_i, c_i), where dn_i is a domain name and c_i is its classification. To create a balanced dataset, two distinct processes were followed to generate the two subsets of TS corresponding to the *Typo* and *Not Typo* classes respectively.

For the *Not Typo* domain class, a set of legitimate benign websites was sampled from the dataset contributed by the authors of Shafi et al. (2024), containing only benign websites. From this set, we randomly sampled 5000 domains from the active and non-parked ones. A domain has been considered *active* if it was registered, and it had a web-server properly replying to HTTP(S) requests at the moment of writing.

To build the *Typo* subset, we started from the list of the top 300 worldwide most frequently searched websites (*TOP*) taken from the DataForSEO dataset (DataForSEO, 2025). DataForSEO provides information on the number of searches performed for each domain in its collection in the last 90 days, categorized by country. From this pool, a subset $S \subset TOP$ consisting of 100 domain names was sampled by selecting one out of every three domains.

For each $d \in S$, a set $PT(d)$ of potentially typosquatted domains was constructed using the open-source tool *ail-typo-squatting* (AIL project, 2025a).

We selected the *ail-typosquatting* (AIL project, 2025a) library over other popular generation tool alternatives such as *typosquatting finder* (AIL project, 2025b) (the webapp of *ail-typosquatting*), *dnstwist* (Elceef, 2025), *urlinsane* (rangertaha, 2025) and *YATT* (Szurdi et al., 2014). This decision was driven by its superior balance of features, ease of use, and competitive performance. While most tools focus on basic typographical errors, *ail-typosquatting* provides much more comprehensive coverage by identifying a wider range of variations, such as look-alike characters and added keywords like "-login". Furthermore, the library is straightforward to set up and its processing speed remains on par with its top competitors.

This process employed the techniques discussed in Section 2 to generate candidate domains with a Damerau–Levenshtein distance 1 from d. From the union of all these sets, we obtained the set $PT = \bigcup_{d \in S} PT(d)$, $|PT| = 108788$. Then, the subset $AD \subseteq PT$ of the active domains, was obtained by filtering PT and keeping only those domains which were found to be active. The cardinality of AD was $|AD| = 21415$, i.e. 19% of generated domains were found to be active at the moment we built the dataset. A set of effective typosquatted domains ET was then built by selecting a subset of AD domains that were manually checked by two distinct people and deemed typosquatted by both of them. Eventually, we selected 6104 typosquatted domains, $|ET| = 6104$. The final *Typo* set has been obtained by taking a random sample of 5000 typosquatted domains from ET.

This way we obtained a balanced dataset TS of 10000 classified domains. For each domain in TS we extracted the informative features described in Section 3.2 and built a feature dataset DS. Both TS and DS are publicly available and can be accessed at Blefari et al. (2025).

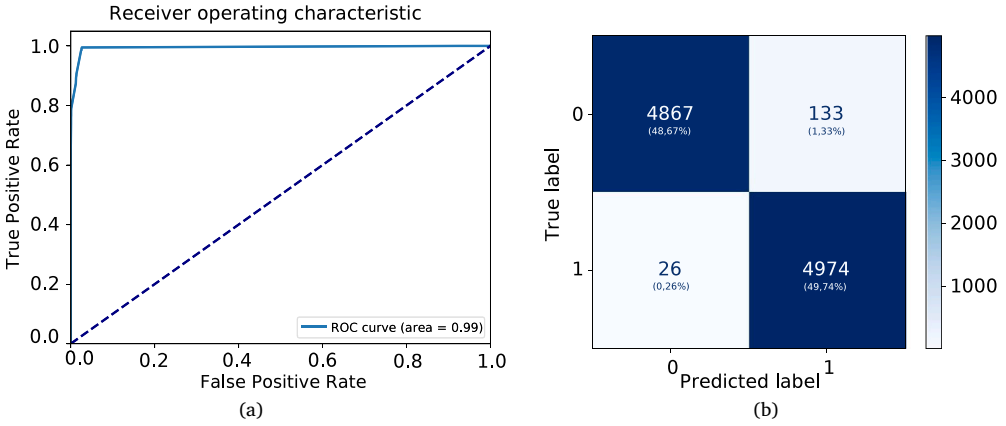


Fig. 4. (a) ROC curve and (b) Confusion matrix for the best threshold ($th^* = 2.1$).

4.4. Performance evaluation

In order to assess the performance of our detection scheme we first selected the optimal value for the threshold th^* by using the ground truth dataset *TS*.

Given a test domain $d \in TS$, let $s(d) = av$ be the ordinal score returned by our algorithm. We consider an aggregation threshold th, for $0 \leq th \leq 7$, and we build a family of binary classifiers each denoted by the two classes $NotTypo_{th} = \{x \in TS \mid s(x) < th\}$, and $Typo_{th} = TS \setminus NotTypo_{th}$.

The corresponding receiver operation characteristic (*ROC*) curve, depicted in Fig. 4(a), shows the trade-off between the True Positive Rate (*TPR*) and False Positive Rate (*FPR*) of each classifier built among the various score thresholds.

The alert value av combines several independent signals of typosquatting: the presence of a candidate target (*CTA*), parking-page evidence (*PARKA*), search-engine ranking divergence (*T10A*), and search-engine correction suggestions (*DYMA*). Because $T10A$ takes values in $[-1.0, 1.0]$ in steps of 0.1, the resulting av takes values in a discrete but non-uniform set (e.g., $2.0, 2.1, \ldots, 6.0$) rather than a set of isolated integers. This allows us to place the threshold at a fractional value to precisely separate borderline cases from clear positives.

We selected th^* by sweeping the threshold over the full range of possible av values and identifying the point that maximizes the TPR/FPR ratio, a criterion that favors high recall while controlling false positives. The optimal threshold is $th^* = 2.1$, for which we have $TPR = 99.48\%$ and $FPR = 2.66\%$.

The choice of $th^* = 2.1$ is well-motivated by the semantics of the alert value. When $CT \neq \emptyset$, the minimum possible alert value is exactly 2: this occurs when $PARKA = 0$, $CTA = 1$, and $T10A_{ct^*} = -1.0$ and $DYMA_{ct^*} = 0$. When $CT = \emptyset$, the alert value is 2 or 3 (depending on $PARKA$). Therefore, the minimum alert value across all non-prefiltered domains is exactly 2. A threshold of 2.1 sits just above this minimum, acting as a *soft lower bound*: it flags any domain that triggers at least one feature, while the fractional offset ensures that even the weakest candidate-target match, where $T10A$ is negative, is still caught.

The *AUC* (Area Under the Curve) is a performance metric used in classification problems to evaluate the ability of a model to distinguish between classes. Given a *ROC* curve, a higher *AUC* indicates better model performance, with 1 representing a perfect classifier and 0.5 indicating a random guess. For our classification scheme we obtained $AUC = 0.99422$.

Fig. 4(b) shows the confusion matrix for $th^* = 2.1$. It can be seen that 9841 over 10000 domains were correctly classified (98.41% of accuracy) by our classification algorithm. The performance metrics of the classifier are reported in Table 7.

Table 7
Performances of TypoDS.

Accuracy	Precision	TPR (Recall)	FPR	F1-score
98.41%	97.40%	99.48%	2.66%	98.43%

Across 100 experimental runs, the average response time of a Google Search API request was measured as 482 ms, with a 95% confidence interval of ±49 ms. The complete execution time of the algorithm was 547 ms ±63 ms, indicating that the majority of the runtime is attributable to the external API request and response latency. Although caching previously retrieved results could substantially reduce this overhead, implementing such a mechanism is considered outside the scope of this work.

4.5. Comparison with ML-based classifiers

To prove the effectiveness of our approach, we made a comparison against other, more complex, ML-based classifiers. Given that the problem we are facing is related to a binary classification (*Typo* or *NotTypo*), we chose to evaluate the most widely used binary classification methods: Support Vector Classifier, Decision Tree, Naive Bayes, Random Forest, Gradient Boosting, Ada Boost, Logistic Regression, and a Neural Network based classifier (Mitchell, 1997; Tan et al., 2005). We evaluated the performances of these algorithms by feeding them with the same informative features evaluated for each domain name (see Section 4).

The comparison has been performed by using the k-fold cross validation technique with $k = 5$ folds. Data has been prepared by sampling the records of *TS* and dividing them into multiple training sets comprising 80% of the data and a test set containing the remaining 20% (Hastie et al., 2009).

Table 8 reports the performance metrics achieved by all the considered classifiers, including TypoDS classifier, and evaluated with a 95% confidence interval. It can be seen that TypoDS outperforms all the other classifiers in terms of precision, accuracy, FPR and F1-score. ADA Boost and Naive Bayes get both an optimal value in terms of recall but perform the worst for the other metrics.

4.6. Comparison with similar technologies

The existence of a typosquatted domain poses a concern both for the owner of the original legitimate domain and for the user who reaches it. When taking the perspective of a legitimate domain owner, there exist some tools for notifying if someone is trying to attempt to divert visits from a given domain. Example of such platforms are

Table 8
Performance evaluation against different classifiers 95% confidence interval.

Classifier	Precision	Recall (TPR)	F1-score	Accuracy	FPR
SVC	$0.9744 \pm 2.96 \cdot 10^{-3}$	$0.9910 \pm 5.34 \cdot 10^{-3}$	$0.9826 \pm 2.03 \cdot 10^{-3}$	$0.9825 \pm 2.01 \cdot 10^{-3}$	$0.0260 \pm 3.17 \cdot 10^{-3}$
Decision Tree	$0.9738 \pm 3.30 \cdot 10^{-3}$	$0.9940 \pm 4.12 \cdot 10^{-3}$	$\mathbf{0.9838 \pm 1.20 \cdot 10^{-3}}$	$\mathbf{0.9836 \pm 1.19 \cdot 10^{-3}}$	$0.0268 \pm 3.56 \cdot 10^{-3}$
Naive Bayes	$0.9557 \pm 8.31 \cdot 10^{-3}$	$\mathbf{1.0000 \pm 0.00 \cdot 10^{-3}}$	$0.9773 \pm 4.35 \cdot 10^{-3}$	$0.9768 \pm 4.55 \cdot 10^{-3}$	$0.0464 \pm 9.11 \cdot 10^{-3}$
Random Forest	$0.9738 \pm 3.30 \cdot 10^{-3}$	$0.9940 \pm 4.12 \cdot 10^{-3}$	$\mathbf{0.9838 \pm 1.20 \cdot 10^{-3}}$	$\mathbf{0.9836 \pm 1.19 \cdot 10^{-3}}$	$0.0268 \pm 3.56 \cdot 10^{-3}$
Gradient Boosting	$0.9738 \pm 3.30 \cdot 10^{-3}$	$0.9940 \pm 4.12 \cdot 10^{-3}$	$\mathbf{0.9838 \pm 1.20 \cdot 10^{-3}}$	$\mathbf{0.9836 \pm 1.19 \cdot 10^{-3}}$	$0.0268 \pm 3.56 \cdot 10^{-3}$
Ada Boost	$0.9557 \pm 8.31 \cdot 10^{-3}$	$\mathbf{1.0000 \pm 0.00 \cdot 10^{-3}}$	$0.9773 \pm 4.35 \cdot 10^{-3}$	$0.9768 \pm 4.55 \cdot 10^{-3}$	$0.0464 \pm 9.11 \cdot 10^{-3}$
Logistic Regression	$0.9739 \pm 3.68 \cdot 10^{-3}$	$0.9930 \pm 6.02 \cdot 10^{-3}$	$0.9834 \pm 1.66 \cdot 10^{-3}$	$0.9832 \pm 1.62 \cdot 10^{-3}$	$0.0266 \pm 3.98 \cdot 10^{-3}$
Neural Network	$0.9737 \pm 3.30 \cdot 10^{-3}$	$0.9936 \pm 4.35 \cdot 10^{-3}$	$0.9836 \pm 1.35 \cdot 10^{-3}$	$0.9834 \pm 1.35 \cdot 10^{-3}$	$0.0268 \pm 3.56 \cdot 10^{-3}$
TypoDS	$\mathbf{0.9940 \pm 4.12 \cdot 10^{-3}}$	$0.9738 \pm 3.30 \cdot 10^{-3}$	$\mathbf{0.9838 \pm 1.20 \cdot 10^{-3}}$	$\mathbf{0.9836 \pm 1.19 \cdot 10^{-3}}$	$\mathbf{0.0061 \pm 4.16 \cdot 10^{-3}}$

Bolster.ai (Bolster.ai, 2025) or Cyberint.com (Cyberint, 2025). These platforms offer a paid service that allows to scan similar domain names. It is checked whether these latter are potentially dangerous and/or whether they try to maliciously exploit the incoming erroneous traffic (e.g. by selling similar item). When a website is deemed dangerous a takedown request can be sent through the platform. The platform tries to notify the problem to the hosting provider, and if the takedown request is denied the request is escalated to multiple Internet Service Providers, Search Engines and to DNS TLD Registries.

TypoDS focuses instead in shielding the final user from potentially dangerous websites reached because of typo errors. Providing users with effective protection against typosquatted domains remains an unresolved challenge, as no comprehensive solution is currently available. A comparison with tools having the same user-centric protection goal follows.

4.6.1. Google Chrome

Google Chrome is one of the most employed browsers in the world and, by its own claim, it has been developed to be security- and privacy-aware by design. Since its release numbered 74, Chrome features a functionality named *lookalike* (Abrams, 2019), aimed to improve the user security by protecting from accessing potentially malicious domain names. The same feature is released in Chromium, the open source version of Google Chrome. The Lookalike feature focuses on detecting:

- Domains that are at a small edit-distance away from other domains (e.g. typo-squatting attempts).
- Domains that use a common word inside the host name (e.g. `common_word.com`).
- Domains that embed other domain names within their own hostname (e.g. `example.com.other_domain.com`).
- Domains that use internationalized domain name (IDN) homographs (e.g. `example.com` where "e" is actually a cyrillic "e").

When Google Chrome Safe Browsing is enabled, the lookalike warnings are displayed informing the user of a potential typing error. Chrome has two levels of confidence, resulting in two different messages for the user. A high-confidence warning is triggered when a lookalike domain is detected, informing the user of the potential danger and requesting confirmation to proceed. A low-confidence warning is a popup informing the user of a possible error, but in this case no confirmation is asked.

The lookalike feature is intended to be a more general approach with respect to TypoDS. However, as stated in the Chromium documentation (Google, 2025), the checks performed by the *lookalike* feature are imperfect and, at the time of writing, the developers of this feature are still working on it. An analysis of Chromium source code shows that the detection of lookalikes is based on an internal whitelist containing the most visited websites, a blacklist, a list of visited websites by the chrome user and a list of common words usually employed for tricking the user (e.g words like *new*, *working*, *dev*). Chrome does not employ the Google search engine in the evaluation process. In order to compare TypoDS and the lookalike feature implemented in Chrome we used an instrumented version of Chrome 122 controlled by a Python script and the Selenium library. This latter allows to control a browser in a programmatic way and to reproduce user activity. In order to compare the Chrome's lookalike feature with TypoDS on typosquatting detection, we used the same dataset described in Section 4.3. For each domain in *TS*, we drove Chrome to make a connection attempt to it and, in case a lookalike warning was raised, we logged this event. Fig. 5 and Table 9 reports the achieved confusion matrix and the performance metrics respectively. Chrome lookalike was not able to recognize most of the typosquatted domains belonging to *TS* ($TPR = 1.14\%$). It can also be noted that it did not have any false positive.

Table 9
Performances of Chrome's *lookalike*.

Accuracy	Precision	TPR(Recall)	FPR	F1-score
50.57%	100.00%	1.14%	0.00%	2.25%

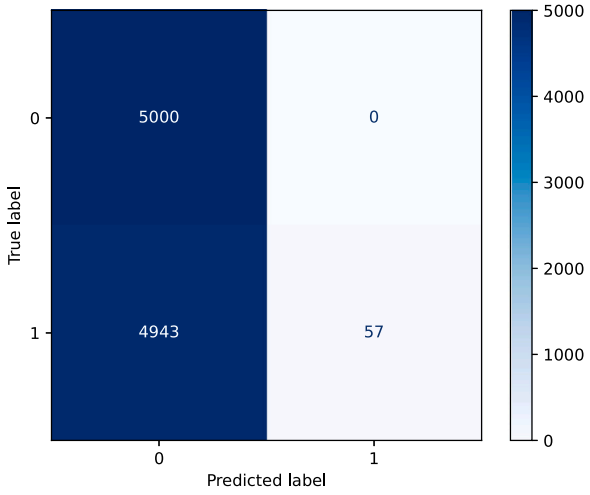


Fig. 5. Confusion Matrix of Google Chrome *lookalike* feature.

4.6.2. Typosquatting protection

We searched the extension stores for the three major browsers at the moment of writing (Google Chrome, Firefox and Microsoft Edge) with the keywords *Typosquatting*, *Lookalike* or *Typo*, and we identified an extension for Google Chrome named Typosquatting Protection (eduard.gm, 2025). This extension is implemented using two lists, a whitelist of about 50 websites and a blacklist, which is initially empty. In this extension, if a candidate domain appears in the whitelist, it is not checked and considered safe. If the domain is in the black list, then it is displayed as a dangerous website. If the domain has a candidate target in the whitelist at a Damerau–Levenshtein distance of 1 it is considered a typo website. Due to a bug, it was not possible to extend the whitelist, so we could not set up an appropriate comparison.

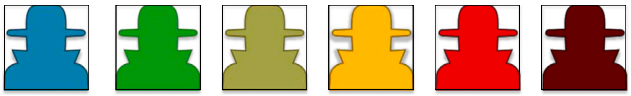


Fig. 6. Different colors of the toggle extension.

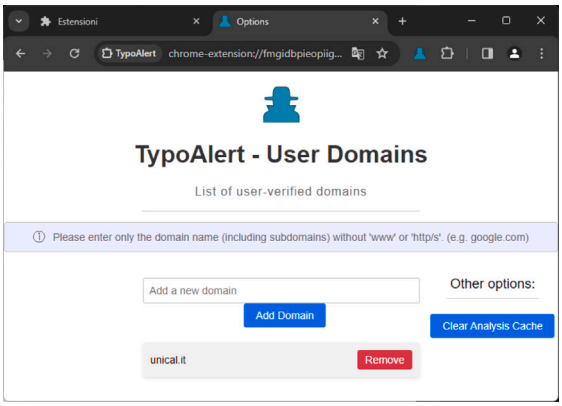
5. The TypoAlert extension for Google Chrome

The availability of a browsing companion software capable of detecting typosquatted sites is essential in order to practically avoid the visit of wrong/dangerous websites. Such a software should feature some characteristics such as detection speed, user friendliness and possibly be transparent and follow least user surprise criteria. To this aim, in our previous work (Blefari et al., 2024), we developed a prototype as an extension for Chrome-based browsers, in which we embedded the TypoDS scheme. To prove the enhancements and optimizations, we updated the prototype, which we will call TypoAlert, according to our TypoDS detection scheme. The concept behind TypoAlert is to perform domain verification in the background and notify users only if they encounter a typosquatted domain. When a website is accessed, TypoAlert considers the fully qualified domain name dn. After evaluation, dn is categorized based on its potential risk level, with the alert value av being determined as outlined in Section 3.3.

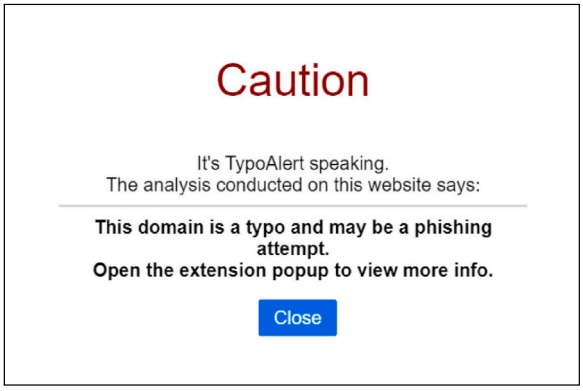
The best score threshold to tell apart *Typo* from *NotTypo* domains is $th^* = 2.1$, as stated in the experiments we conducted and detailed in Section 4. Based on this threshold value, we identified five severity levels and we mapped the numerical alert values to mnemonic names accordingly (see algorithm 4). Each mnemonic name has been then related to a different colored icon. In browsers with TypoAlert installed, this icon will appear as a new toggle visible on the upper right side of the browser (see Fig. 6). The toggle provides users with a rapid and non-invasive visual response regarding the classification for the current domain name dn which is going to be displayed in the current opened tab. The chosen color categorization is as follows:

- Blue: if the analysis is not started yet;
- Dark-Red: if dn is marked as *TypoMalware*;
- Red: dn is marked as *Typo*;
- Yellow: if dn is marked as *ProbablyTypo*;
- Green-Yellow: if dn is marked as *ProbablyNotTypo*;
- Green: if dn is marked as *NotTypo*.

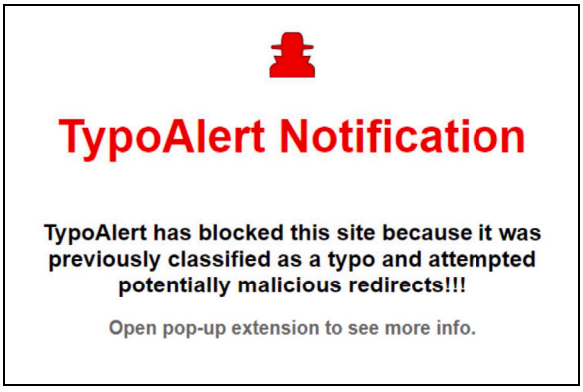
If the extension's analysis indicates a label among *ProbablyTypo*, *Typo* or *TypoMalware* (colors from yellow to dark red), an alert appears, warning the user about the detected severity level. For the Green and Green-Yellow severity levels, no alert is given and the user is allowed to visit the related web page. An example is depicted in Fig. 7(b). Domain names classified as typosquattings are retained in the extension cache: if a domain d has been classified as typosquatted in the last 30 days, the web page of d is blocked and replaced by a notification page as depicted in Fig. 7(c), Users can always access the extension's options and add misclassified domains to the verified user whitelist, excluding them from the analysis (Fig. 7(a)).



(a) Option page



(b) Alert popup



(c) Notification page

Fig. 7. TypoAlert GUI elements.

Algorithm 4 Labeling Algorithm

Input: av alert value
Output: classification label

```
if av = 0 then
    return "NotTypo"
else if av = 10 then
    return "TypoMalware"
else if av < 2.1 then
    return "ProbablyNotTypo
else if av ≥ 2.1 then
    return "ProbablyTypo"
else
    return "Typo"
end if
```

Despite the encouraging results, TypoAlert has some limitations. First, its effectiveness depends on that of TypoDS, which in turn relies on the freshness and completeness of the maintained whitelists and blacklists. Consequently, an appropriate update policy must be established to ensure that these lists remain effective against newly

registered or compromised domains. Second, several features depend on external services, such as search engine APIs, whose availability, latency, quota limitations, or changes in ranking policies may affect both the responsiveness and the overall effectiveness of the detection process. Third, the use of commercial search engine APIs may incur operational costs, especially when processing a large number of requests. As a result, the scalability of the proposed approach may be constrained by the pricing model and usage quotas imposed by the service provider. The Chrome extension was further tested to assess the impact of the browser-based deployment on execution performance. Across 100 experimental runs, the average response time of the Google Search API request was measured as 491 ms, with a 95% confidence interval of ±52 ms. The complete execution time of the extension, including API communication, browser-side processing, and interaction with the web page, was 592 ms with a 95% confidence interval of ±71 ms. Compared to the standalone algorithm, the extension introduces only a marginal overhead of a few milliseconds, mainly due to the additional communication layers between the browser environment and the extension components. These results confirm that the external API request and response latency remain the dominant contributor to the overall execution time. Although caching mechanisms could further reduce this latency, their implementation is considered outside the scope of this work.

6. Conclusions and future work

In this work we have described TypoDS, a typosquatting detection scheme which exploits some informative features extracted from domain names by leveraging search engines' knowledge. While the proposed scheme does not relies on machine learning algorithms it shows very good performance. In a direct comparison with state-of-art ML classification algorithms, it slightly outperforms almost all of them in terms of Precision, Accuracy and False Positive Rate. It should be noted that ML algorithms were not fed with raw data but with the features we purposely devised for TypoDS and require a training phase. We overcame the absence of a ground-truth by manually building it from scratch. All the datasets are freely available on request for further researches on this topic. We embedded TypoDS into an extension, named TypoAlert, for the Chrome Web browser, which is expected to revitalize the not-so-live landscape of user-centric initiatives against typosquatting. Future research directions include: *(i)* enhancing TypoDS with features that tackle typosquatting from an even more user-centric perspective, in the spirit of dynamic skins (Dhamija and Tygar, 2005); *(ii)* evaluating the benefits of including phishing detection based on perceptual-hashing (Klinger and Starkweather, 2021); *(iii)* investigating about the adaptation LLM models by tailoring them for typosquat detection. TypoAlert is freely available at https://github.com/cybers3clab/typoalert_webapp_pub.

CRediT authorship contribution statement

Francesco Blefari: Software, Methodology, Conceptualization. **Angelo Furfaro:** Software, Methodology, Conceptualization. **Giovambattista Ianni:** Software, Methodology, Conceptualization. **Francesco Aurelio Pironti:** Software, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors are grateful to Alessandro Viscomi for the first implementation of TypoAlert.

The work of Angelo Furfaro and Giovambattista Ianni was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

The work of Giovambattista Ianni was also partially supported by project FAIR (PE0000013) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

The work of Francesco A. Pironti was supported by Agenzia per la cybersicurezza nazionale under the 2024–2025 funding programme for promotion of XL cycle PhD research in cybersecurity (CUP H23C24000640005).

Data availability

The TypoDS validation dataset can be downloaded from https://github.com/cybers3clab/typosquatting_dataset.

References

Abrams, L., 2019. Chrome to display warnings about similar or lookalike URLs. https://www.bleepingcomputer.com/news/software/chrome-to-display-warnings-about-similar-or-lookalike-urls/.

abuse.ch, 2025. Urlhaus | malware URL exchange. URL https://urlhaus.abuse.ch/.

Agten, P., Joosen, W., Piessens, F., Nikiforakis, N., 2015. Seven months' worth of mistakes: A longitudinal study of typosquatting abuse. In: Proceedings 2015 Network and Distributed System Security Symposium. Internet Society, San Diego, California,USA, http://dx.doi.org/10.14722/ndss.2015.23058.

Ahmad, I., Parvez, M.A., Iqbal, A., 2019. TypoWriter: A tool to prevent typosquatting. COMPSAC, In: 2019 IEEE 43rd Annual Computer Software and Applications Conference, vol. 1, pp. 423–432. http://dx.doi.org/10.1109/COMPSAC.2019.00068.

AIL project, 2025a. Ail-typo-squatting. URL https://github.com/typosquatter/ail-typo-squatting.

AIL project, 2025b. Typosquatting finder. URL https://typosquatting-finder.circl.lu/.

Anvilogic, 2025. Anvilogic/T5-typosquat-training-dataset | datasets at hugging face. URL https://huggingface.co/datasets/Anvilogic/T5-Typosquat-Training-Dataset.

Banerjee, A., Barman, D., Faloutsos, M., Bhuyan, L.N., 2008. Cyber-fraud is one typo away. In: IEEE INFOCOM 2008 - the 27th Conference on Computer Communications. pp. 1939–1947. http://dx.doi.org/10.1109/INFOCOM.2008.258.

Banerjee, A., Rahman, M.S., Faloutsos, M., 2011. SUT: Quantifying and mitigating URL typosquatting. Comput. Netw. 55 (13), 3001–3014. http://dx.doi.org/10.1016/j.comnet.2011.06.005.

Blefari, F., Furfaro, A., Ianni, G., Pironti, F.A., 2025. TypoDS validation dataset. https://github.com/cybers3clab/typosquatting_dataset.

Blefari, F., Furfaro, A., Ianni, G., Viscomi, A., 2024. Combining anti-typosquatting techniques. In: Stefanidis, K., Systä, K., Matera, M., Heil, S., Kondylakis, H., Quintarelli, E. (Eds.), Web Engineering. Springer Nature Switzerland, Cham, pp. 246–254. http://dx.doi.org/10.1007/978-3-031-62362-2_17.

Bolster.ai, 2025. Bolster | AI Security, Domain Monitoring & Takedowns. URL https://bolster.ai/.

Boutin, P., 2005. The typo millionaires. Slate. URL https://slate.com/technology/2005/02/how-to-kill-off-the-oldest-scam-on-the-internet.html.

Chen, G., Johnson, M.F., Marupally, P.R., Singireddy, N.K., Yin, X., Paruchuri, V., 2009. Combating typo-squatting for safer browsing. In: 2009 International Conference on Advanced Information Networking and Applications Workshops. pp. 31–36. http://dx.doi.org/10.1109/WAINA.2009.98.

Cyberint, 2025. Cyberint - threar intelligence & digital risk protection. URL https://cyberint.com/.

Damerau, F.J., 1964. A technique for computer detection and correction of spelling errors. Commun. ACM 7 (3), 171–176. http://dx.doi.org/10.1145/363958.363994.

DataForSEO, 2025. Top 1000 websites by ranking keywords. URL https://dataforseo.com/free-seo-stats/top-1000-websites.

Dhamija, R., Tygar, J.D., 2005. The battle against phishing: Dynamic security skins. In: Cranor, L.F. (Ed.), Proceedings of the 1st Symposium on Usable Privacy and Security, SOUPS 2005. In: ACM International Conference Proceeding Series, vol. 93, ACM, Pittsburgh, Pennsylvania, USA, pp. 77–88. http://dx.doi.org/10.1145/1073001.1073009.

Edelman, B., 2003. Large-scale registration of domains with typographical errors. Harvard University. https://cyber.harvard.edu/archived_content/people/edelman/typo-domains/.

eduard.gm, 2025. Typosquatting protection. URL https://chromewebstore.google.com/detail/typosquatting-protection/ofhhhkhomfdapknngpeefhpipfcgmkee?pli=1.

Elceef, 2025. Elceef/dnstwist: Domain name permutation engine for detecting homograph phishing attacks, typo squatting, and brand impersonation. URL https://github.com/elceef/dnstwist.

Google, 2025. "Lookalike" Warnings in Google Chrome. URL https://chromium.googlesource.com/chromium/src/+/master/docs/security/lookalikes/lookalike-domains.md.

Hamming, R.W., 1950. Error detecting and error correcting codes. Bell Syst. Tech. J. 29 (2), 147–160. http://dx.doi.org/10.1002/j.1538-7305.1950.tb00463.x.

Hastie, T., Tibshirani, R., Friedman, J., 2009. The elements of statistical learning: Data mining, inference, and prediction. In: Springer series in statistics, Springer, New York, http://dx.doi.org/10.1007/978-0-387-84858-7.

Interisle Consulting Group, 2025. Cybercrime information center. URL https://www.cybercrimeinfocenter.org/records.

Jaro, M.A., 1989. Advances in record-linkage methodology as applied to matching the 1985 census of tampa, florida. J. Amer. Statist. Assoc. 84 (406), 414–420. http://dx.doi.org/10.1080/01621459.1989.10478785, arXiv:https://doi.org/10.1080/01621459.1989.10478785.

Joon Sern, L., Gui Peng David, Y., 2021. TypoSwype: An imaging approach to detect typo-squatting. In: 2021 11th IFIP International Conference on New Technologies, Mobility and Security. NTMS, pp. 1–5. http://dx.doi.org/10.1109/NTMS49979.2021.9432673.

Khan, M.T., Huo, X., Li, Z., Kanich, C., 2015. Every second counts: Quantifying the negative externalities of cybercrime via typosquatting. In: 2015 IEEE Symposium on Security and Privacy. pp. 135–150. http://dx.doi.org/10.1109/SP.2015.16.

Klinger, E., Starkweather, D., 2021. pHash: The open source perceptual hash library. URL https://www.phash.org/docs/.

Koide, T., Fukushi, N., Nakano, H., Chiba, D., 2023. PhishReplicant: A language model-based approach to detect generated squatting domain names. In: Proceedings of the Annual Computer Security Applications Conference. http://dx.doi.org/10.1145/3627106.3627111.

Levenshtein, V.I., 1965. Binary codes capable of correcting deletions, insertions, and reversals. Sov. Phys. Dokl. 10, 707–710, URL https://api.semanticscholar.org/CorpusID:60827152.

Mercer, J., 2000. Cybersquatting: Blackmail on the information superhighway. "Journal Sci. Technol. Law" URL https://www.bu.edu/law/journals-archive/scitech/volume6/mercer.pdf.

Mitchell, T.M., 1997. Machine Learning. (9), McGraw-hill, New York, USA.

Moore, T., Edelman, B., 2010. Measuring the perpetrators and funders of typosquatting. In: Sion, R. (Ed.), Financial Cryptography and Data Security. Springer, Berlin, Heidelberg, pp. 175–191.

Navarro, G., 2001. A guided tour to approximate string matching. ACM Comput. Surv. 33 (1), 31–88. http://dx.doi.org/10.1145/375360.375365.

Neupane, S., Holmes, G., Wyss, E., Davidson, D., De Carli, L., 2023. Beyond typosquatting: an in-depth look at package confusion. In: Proc. of the 32nd USENIX Conference on Security Symposium. SEC '23, USENIX Association, USA, pp. 3439–3456, URL https://www.usenix.org/system/files/usenixsecurity23-neupane.pdf.

rangertaha, (2025).URL https://github.com/rangertaha/urlinsane.

Raymond, E.S., 2008. The art of UNIX programming. In: Addison-Wesley professional computing series, Addison-Wesley, Boston.

Ronda, T., Saroiu, S., Wolman, A., 2008. Itrustpage: a user-assisted anti-phishing tool. SIGOPS Oper. Syst. Rev. 42 (4), 261–272. http://dx.doi.org/10.1145/1357010.1352620.

Shafi, M., Lashkari, A.H., Mohanty, H., 2024. Unveiling malicious DNS behavior profiling and generating benchmark dataset through application layer traffic analysis. Comput. Electr. Eng. 118, 109436. http://dx.doi.org/10.1016/j.compeleceng.2024.109436, URL https://www.sciencedirect.com/science/article/pii/S0045790624003641.

signalblur, 2025. ScumBots-DataFeed. URL https://github.com/signalblur/ScumBots-DataFeed.

Spaulding, J., Upadhyaya, S., Mohaisen, A., 2016. The landscape of domain name typosquatting: Techniques and countermeasures. In: 2016 11th International Conference on Availability, Reliability and Security. ARES, pp. 284–289. http://dx.doi.org/10.1109/ARES.2016.84.

Stampar, M., 2025. BlackBook: a historical (black)list of malicious domains. URL https://github.com/stamparm/blackbook.

Szurdi, J., Kocso, B., Cseh, G., Spring, J., Felegyhazi, M., Kanich, C., 2014. The long "Taile" of typosquatting domain names. In: 23rd USENIX Security Symposium (USENIX Security 14). USENIX Association, San Diego, CA, pp. 191–206, URL.

Tan, P.-N., Steinbach, M., Kumar, V., 2005. Introduction to Data Mining, (First Edition). Addison-Wesley Longman Publishing Co. Inc., USA.

Taylor, M., Vaidya, R., Davidson, D., De Carli, L., Rastogi, V., 2020. Defending against package typosquatting. In: Network and System Security: 14th International Conference, NSS 2020, Melbourne, VIC, Australia, November 25–27, 2020, Proceedings. Springer-Verlag, Berlin, Heidelberg, pp. 112–131. http://dx.doi.org/10.1007/978-3-030-65745-1_7.

ViriBack, 2025. ViriBack blog. URL https://viriback.com/.

Wagner, R.A., Fischer, M.J., 1974. The string-to-string correction problem. J. ACM 21 (1), 168–173. http://dx.doi.org/10.1145/321796.321811.

Wang, Y.-M., Beck, D., Wang, J., Verbowski, C., Daniels, B., 2006. Strider Typo-Patrol: Discovery and analysis of systematic Typo-Squatting. In: 2nd Workshop on Steps To Reducing Unwanted Traffic on the Internet. SRUTI '06, http://dx.doi.org/10.5555/1251296.1251301.

Welch, J., 2025. Training large language models for advanced typosquatting detection. arXiv:2503.22406.

WhoisXMLAPI, 2025. Whoisxmlapi | typosquatting data feed. URL https://typosquatting.whoisxmlapi.com/.